

FluidDynamicsMetal

Equations and real-time approximations

How the Swift renderer and Metal shaders create moving dye

Source snapshot: 9477dabc41a2 | 26 September 2026

What this simulation represents

The app evolves a two-dimensional velocity field and a scalar dye field on a rectangular grid. Pointer or touch motion adds local velocity; contact also adds dye. Advection carries both fields, an artificial swirl force restores visible rotation, and a pressure-like correction reduces compression and expansion. The final image is a colour mapping of one stored field.

This is a visual fluid model inspired by incompressible flow. It does not model a water surface, three-dimensional smoke, variable mass density, or calibrated physical units. In particular, the field called *Density* is transported dye, not the fluid's mass density.

Reading rule. The source code is the authority throughout this document. Continuum equations explain the intended model; discrete equations describe the actual shader operations. Consequences inferred from those operations are identified as numerical limitations, not as measured errors.

Reading map

Page	Topic
2	The underlying fluid and dye equations
3	Grid coordinates, stored fields, and the exact frame sequence
4	Backward advection and bilinear interpolation
5	Contact forces, dye injection, and fading
6	Vorticity confinement and boundaries
7	Pressure projection and the 40-step Jacobi solve
8	Why incompressibility and conservation are approximate
9	GPU execution, precision, initial state, and resizing
10	Displayed fields and the actual tuning parameters
11	Source map, scope of verification, and rebuilding this PDF

The shared simulation sources compile into both the Mac and iOS apps. Platform input handling differs, but the numerical shaders and frame pipeline described here are shared. This document is self-contained; external reading is optional.

1 From fluid mechanics to a visual model

Let $\mathbf{u} = (u, v)$ be velocity, p physical pressure, ρ_0 constant fluid mass density, ν kinematic viscosity, and $\mathbf{f}$ force per unit mass. The incompressible Navier–Stokes model in two dimensions is

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u} = -\frac{1}{\rho_0} \nabla p + \nu \nabla^2 \mathbf{u} + \mathbf{f}, \quad (1)$$

$$\nabla \cdot \mathbf{u} = 0. \quad (2)$$

The material derivative on the left follows a moving fluid parcel. Pressure redistributes velocity to enforce the zero-divergence constraint. Viscosity diffuses momentum, smoothing spatial velocity differences.

For a passive dye concentration c , a useful companion equation is

$$\frac{\partial c}{\partial t} + \mathbf{u} \cdot \nabla c = \kappa \nabla^2 c + s - \lambda_c c. \quad (3)$$

Here κ is dye diffusivity, s is dye injection, and λ_c controls decay. “Passive” means c does not feed back into the momentum equation. The app has no buoyancy term coupling dye to velocity.

Which parts are actually implemented?

Continuum idea	Actual implementation
Transport ($\mathbf{u} \cdot \nabla$)	Backward tracing with a single velocity sample, then bilinear interpolation of the transported field.
Pressure constraint	Central-difference divergence, 40 Jacobi iterations, then central-difference pressure-gradient subtraction.
Viscosity $\nu \nabla^2 \mathbf{u}$	No explicit viscosity or momentum-diffusion solve.
Dye diffusion $\kappa \nabla^2 c$	No explicit dye-diffusion solve.
Forcing and source	Gaussian velocity increments and Gaussian dye deposits around user contacts.
Small-scale rotation	Vorticity confinement, an added visual force.
Decay	The same multiplicative retention factor applied to velocity and dye during their advection passes.

Consequently, the closest continuum interpretation is an inviscid, incompressible model with artificial damping and confinement, plus passive dye. Interpolation itself also smooths fields. That *numerical diffusion* is not a chosen physical viscosity and should not be assigned a viscosity value.

There is no time measured in seconds in the solver. Using grid cells as length units and one advancing frame as a time unit gives effective $h = 1$ and $\Delta t = 1$. Stored velocity is therefore best understood as a displacement in cells per simulation step. This convention also absorbs the physical pressure and density scale into the pressure-correction field (page 7).

Source: `Renderer.draw(in:); Shaders.metal: advect, jacobi, gradient, and vorticityConfinement.`

2 The grid and the exact order of a frame

For view bounds measured in platform points, the renderer chooses

$$W = \left\lfloor \frac{\text{bounds.width}}{s_g} \right\rfloor, \quad H = \left\lfloor \frac{\text{bounds.height}}{s_g} \right\rfloor, \quad s_g = 1. \quad (4)$$

s_g is `ScreenScaleAdjustment`. The grid uses view bounds, not the drawable's Retina pixel dimensions. All quantities are *collocated*: both velocity components and each scalar live at the same texel centres. For $0 \leq i < W$, $0 \leq j < H$,

$$\mathbf{x}_{ij} = (i + \tfrac{1}{2}, j + \tfrac{1}{2}), \quad \boldsymbol{\xi}_{ij} = \left(\frac{i + 1/2}{W}, \frac{j + 1/2}{H} \right), \quad \text{offsets} = (1/W, 1/H). \quad (5)$$

Thus a normalized-texture offset of $1/W$ is one grid cell, not a physical derivative scale. The texture convention has x rightwards and y downwards. Mac input flips its vertical coordinate to match this convention.

Slab	First channel	Second channel
Velocity	Horizontal velocity u	Vertical velocity v
Density	Dye c	Carried but not used for dye display
Divergence	Discrete divergence b	Written as zero
Vorticity	Signed curl ω	Written as zero
Pressure	Correction potential ϕ	Written as zero

An advancing frame, in source order

Let $A_{\mathbf{a}}(q)$ mean the advection-and-retention operation on page 4. Superscripts a, b label intermediate fields, not time derivatives.

1. Advect velocity with itself: $\mathbf{u}^a = A_{\mathbf{u}^n}(\mathbf{u}^n)$.
2. Advect dye with that *updated* velocity: $c^a = A_{\mathbf{u}^a}(c^n)$.
3. For every contact batch, add velocity and dye splats, giving $\mathbf{u}^b$ and c^{n+1} .
4. Compute $\omega = \text{curl}_h \mathbf{u}^b$ and store it.
5. Add confinement to $\mathbf{u}^b$, then zero the outer velocity cells, giving the provisional velocity $\mathbf{w}$.
6. Compute $b = D_h \mathbf{w}$; perform exactly 40 pressure iterations.
7. Set $\mathbf{u}^{n+1} = \mathbf{w} - G_h \phi$ and zero outer velocity cells again.
8. Display the selected stored field, if a drawable is available.

This is sequential *operator splitting*. A dye deposit is not transported until the next frame, and the current frame's force and pressure correction do not enter the current dye backtrace. These are visible timing choices, not simultaneous solutions of equations (1)–(3). When paused or inactive, `draw(in:)` skips all evolution and can display an existing field without advancing it.

Source: `Renderer.swift`: `resize` callback, `draw(in:)`, and the slab pass wrappers; `Shaders.metal`: `BufferData`.

3 Advection: follow a parcel backwards

Ignoring sources and decay, a transported quantity satisfies $\partial_t q + \mathbf{u} \cdot \nabla q = 0$: it is constant along a fluid trajectory. Rather than move samples forward and leave holes, the shader asks where the parcel arriving at each destination cell came from.

For a destination centre $\mathbf{x}$, it uses one explicit Euler step backwards along the trajectory (a first-order backtrace),

$$\mathbf{x}_d = \mathbf{x} - \Delta t \mathbf{a}(\mathbf{x}), \quad A_{\mathbf{a}}(q)(\mathbf{x}) = r \mathcal{I}[q](\mathbf{x}_d), \quad \Delta t = 1, \quad (6)$$

where r is retention and $\mathcal{I}$ is bilinear interpolation. This is a semi-Lagrangian method: a fixed (Eulerian) grid samples approximate moving (Lagrangian) trajectories. Velocity at the destination is sampled with nearest filtering; the backtraced field uses four manually interpolated samples.

The interpolation actually in `bilerpFrag`

For $\mathbf{x}_d = (x_d, y_d)$, define

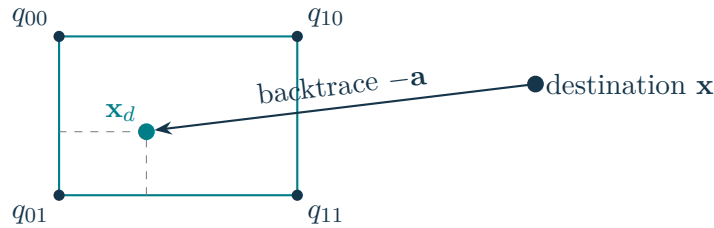
$$(x_0, y_0) = \lfloor \mathbf{x}_d - (\tfrac{1}{2}, \tfrac{1}{2}) \rfloor + (\tfrac{1}{2}, \tfrac{1}{2}), \quad (7)$$

$$a = x_d - x_0, \quad b = y_d - y_0, \quad 0 \leq a, b < 1. \quad (8)$$

With $q_{00}, q_{10}, q_{01}, q_{11}$ at the surrounding texel centres,

$$\mathcal{I}[q] = (1-a)(1-b)q_{00} + a(1-b)q_{10} + (1-a)bq_{01} + abq_{11}. \quad (9)$$

The formula applies independently to both channels. The half-cell offset is essential: integer coordinates lie between texel centres.



Schematic interpolation cell; the departure point can lie several cells away.

Why this is useful, and what it costs

The interpolation weights are nonnegative and sum to one. In exact arithmetic, this sampling step with $0 \leq r \leq 1$ cannot increase the maximum absolute value of any sampled component. This avoids the usual explicit-advection stability restriction that a parcel move less than roughly one cell per step.

It does not make large steps accurate. A straight backtrace ignores trajectory curvature, repeated interpolation blurs fine dye filaments and vortices, and the method does not conserve the sum of dye values. No higher-order tracing, MacCormack/BFECC correction, or conservative flux transport is present. The stability argument applies to this sampling operation, not to the whole solver with forces, finite precision, and approximate projection.

Source: `Shaders.metal`: `advect` and `bilerpFrag`.

4 User input and the meaning of Fade

For a contact at grid position $\mathbf{z}_k$, the shader uses

$$g_k(\mathbf{x}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{z}_k\|^2}{R}\right), \quad \mathbf{u} \leftarrow \mathbf{u} + \sum_k \mathbf{I}_k g_k, \quad c \leftarrow c + \sum_k d g_k. \quad (10)$$

$\mathbf{I}_k = F \Delta \mathbf{p}_k / s_g$ is the supplied view-space displacement multiplied by Force and converted to grid units; d is Dye. The contact position is likewise $\mathbf{z}_k = \mathbf{p}_k / s_g$. There is no division by elapsed event time, no integration of force over seconds, and no normalization of the Gaussian's total integral.

Despite its name, `inkRadius` is the denominator R , not a radius or a standard deviation. The e^{-1} distance is $\sqrt{R}$, and the equivalent Gaussian standard deviation is $\sqrt{R/2}$. At the default $R = 150$, these are about 12.25 and 8.66 grid cells. Increasing R also increases total deposited dye: on an infinite continuous plane, $\int g dA = \pi R$.

For mouse input the active renderer supplies $R = 150/s_g$. For touch input it uses $R = 150(\min(\text{view width}, \text{view height})/375)^2$, with the short side clamped to at least one point. At the current $s_g = 1$, the touch footprint therefore scales with screen size. `SimulationTuning.radius` exists but is not read by this active path.

Contacts occupy ten uniform slots per pass; extra contacts are split into additional batches, each with a velocity pass and a dye pass. An unused slot is marked by position (0,0); `writeContacts` nudges a retained origin contact to (0.01,0.01) before grid scaling. Queued taps have zero impulse but still deposit dye. Mac drag increments are measured between draws. iOS supplies event-to-event displacements; the renderer reuses the latest touch contacts until an input update replaces them. Input is consequently frame/event-rate dependent rather than a calibrated force measurement.

Retention is applied to both transported fields

The scalar r in equation (6) multiplies *both* self-advected velocity and advected dye. Fade therefore damps motion as well as colour. For an isolated value affected only by retention, in exact arithmetic,

$$q_n = r^n q_0, \quad \lambda_{\text{step}} = -\ln r, \quad n_{1/2} = \frac{\ln(1/2)}{\ln r}. \quad (11)$$

The nominal default $r = 0.998$ gives a half-life of about 346 steps, or 5.77 seconds if every second contains 60 advancing frames. This is an illustrative decay calculation, not a measured dye lifetime: transport, sources, rounding, and display saturation change what is visible. The shader casts r to `half`; 0.998 becomes approximately 0.998046875 even before repeated output rounding.

The view requests 60 frames per second, but the solver does not compensate for missed frames. At 30 advancing frames per second, decay measured in seconds roughly slows by a factor of two. Multiplicative damping is proportional to the local value; physical viscosity instead depends on spatial curvature.

Source: `Renderer: writeContacts, nextBuffer, input methods; platform RenderViewControllers; gaussSplat, applyForceVector, applyForceScalar, and advect.`

5 Vorticity confinement and wall treatment

Vorticity measures local rotation. In the shader’s coordinate convention,

$$\omega = \partial_x v - \partial_y u, \quad \omega_{ij} = \frac{1}{2}(v_{i+1,j} - v_{i-1,j} - u_{i,j+1} + u_{i,j-1}). \quad (12)$$

This is a central difference with unit grid spacing. The visual interpretation of its sign must respect the downward texture y direction.

Semi-Lagrangian interpolation weakens small vortices. Confinement adds a force constructed from variation in the *magnitude* of vorticity, intended to retain more rotational detail. Define

$$\boldsymbol{\eta}_{ij} = \frac{1}{2} \begin{pmatrix} |\omega_{i+1,j}| - |\omega_{i-1,j}| \\ |\omega_{i,j+1}| - |\omega_{i,j-1}| \end{pmatrix}, \quad (13)$$

$$\mathbf{f}_{\text{conf}} = \frac{S \omega_{ij}}{\sqrt{\max(\epsilon, \|\boldsymbol{\eta}_{ij}\|^2)}} \begin{pmatrix} \eta_y \\ -\eta_x \end{pmatrix}, \quad \epsilon = 2.4414 \times 10^{-4}, \quad (14)$$

$$\mathbf{w} = B(\mathbf{u}^b + \mathbf{f}_{\text{conf}}). \quad (15)$$

S is Swirl (default 0.4), and B is the boundary overwrite below. In code, the two gradient components are constructed in reversed order and the second force component is negated. Equation (14) is that exact algebra, equivalent to a regularized two-dimensional form of $\mathbf{N} \times \boldsymbol{\omega}$. The denominator clamps the *squared* norm before taking its square root; it is not $\|\boldsymbol{\eta}\| + \epsilon$.

This force is artificial. It adds rotational motion but does not reconstruct eddies lost below grid resolution or conserve kinetic energy. Uniform $|\omega|$ gives zero force; a large Swirl setting is not a physical turbulence parameter. The local shader time multiplier is explicitly 1.0.

Two distinct boundary mechanisms

First, simulation samplers specify nearest filtering but omit addressing and coordinate modes. Metal’s defaults are normalized coordinates and `clamp_to_edge` (Apple MSL specification, section 2.10, table 2.7). Samples beyond an edge thus reuse edge texels. Backtraces do not wrap around the domain; the outer pressure neighbour is effectively the edge pressure itself.

Second, confinement and final gradient subtraction explicitly set *both* velocity components to zero wherever

$$x \leq 1 \vee y \leq 1 \vee x \geq W - 1 \vee y \geq H - 1. \quad (16)$$

At texel centres on an ordinary grid this is the outermost row/column, a no-slip-like border. The other passes do not enforce this overwrite, and dye is not explicitly zeroed at walls. Edge clamping and this velocity mask are a pragmatic boundary treatment, not a consistent ghost-cell discretization of all wall conditions. The final overwrite can itself alter nearby divergence.

Source: `Shaders.metal`: `vorticity`, `vorticityConfinement`, `gradient`, and sampler declarations. External default: Apple MSL specification.

6 Pressure as a velocity correction

Advection and forcing generally leave provisional velocity $\mathbf{w}$ with nonzero divergence. The ideal projection subtracts a gradient,

$$\mathbf{u}^{n+1} = \mathbf{w} - \nabla\phi, \quad \nabla^2\phi = \nabla \cdot \mathbf{w}. \quad (17)$$

Taking the divergence of the first equation derives the second if the result is to satisfy $\nabla \cdot \mathbf{u}^{n+1} = 0$. In physical units one could set $\phi = (\Delta t / \rho_0)p$. Here the code stores the scaled correction directly, with unit step and cell conventions; it is not pressure in pascals.

Step 1: form the right-hand side

For $\mathbf{w} = (w^x, w^y)$, the `divergence` shader computes

$$b_{ij} = D_h \mathbf{w} = \frac{1}{2} (w_{i+1,j}^x - w_{i-1,j}^x + w_{i,j+1}^y - w_{i,j-1}^y). \quad (18)$$

b is stored once for the pressure loop. Positive divergence means local outflow exceeds inflow under this discrete measure.

Step 2: approximately solve a five-point Poisson equation

The pressure iteration targets

$$L_5\phi = \phi_{i-1,j} + \phi_{i+1,j} + \phi_{i,j-1} + \phi_{i,j+1} - 4\phi_{ij} = b_{ij}. \quad (19)$$

Rearranging for the centre value gives the Jacobi update used by `jacobi`:

$$\boxed{\phi_{ij}^{(k+1)} = \frac{\phi_{i-1,j}^{(k)} + \phi_{i+1,j}^{(k)} + \phi_{i,j-1}^{(k)} + \phi_{i,j+1}^{(k)} - b_{ij}}{4}} \quad k = 0, \dots, 39. \quad (20)$$

This explains the shader constants `alpha = -1` and `beta = 4`. All cells read iteration k from one texture and write iteration $k + 1$ to another. This is Jacobi, not Gauss–Seidel, because no newly computed neighbour is read in the same iteration.

The renderer retains the previous pressure texture between frames: it is a warm start, not a fresh zero guess on every frame. The loop is literally `0..<40; SimulationTuning.pressureIterations` is not consulted. There is no residual-based stopping criterion.

Step 3: subtract the central-difference gradient

$$G_h\phi = \frac{1}{2} \begin{pmatrix} \phi_{i+1,j} - \phi_{i-1,j} \\ \phi_{i,j+1} - \phi_{i,j-1} \end{pmatrix}, \quad \mathbf{u}^{n+1} = B(\mathbf{w} - G_h\phi^{(40)}). \quad (21)$$

The pressure display reads this last iterate. It is a useful diagnostic of the correction pattern, but does not establish a converged physical pressure field.

Source: `Shaders.metal: divergence, jacobi, gradient; Renderer.draw(in:): pressure loop.`

7 Why the result is only approximately incompressible

There are two independent reasons the pressure operation does not guarantee a zero-divergence grid velocity. First, 40 Jacobi steps only approximate a solve. Second, even a perfectly converged solve uses a Laplacian that does not equal the composition of the actual divergence and gradient operators.

A stencil mismatch visible directly in the shaders

Away from boundaries, composing the two central differences gives

$$D_h G_h \phi = \frac{\phi_{i+2,j} + \phi_{i-2,j} + \phi_{i,j+2} + \phi_{i,j-2} - 4\phi_{ij}}{4}. \quad (22)$$

This reaches neighbours *two* cells away, whereas L_5 in the Jacobi solve reaches neighbours one cell away. Both approximate a continuum Laplacian on smooth fields, but they are different discrete operators.

Define the algebraic residual after the pressure solve as $\mathcal{R} = b - L_5 \phi$. Before the final boundary overwrite, equations (18) and (21) imply

$$D_h(\mathbf{w} - G_h \phi) = \underbrace{\mathcal{R}}_{\text{incomplete solve}} + \underbrace{(L_5 - D_h G_h)\phi}_{\text{operator mismatch}}. \quad (23)$$

Thus more pressure iterations alone do not remove all measured divergence. For an interior checkerboard $\phi_{ij} = (-1)^{i+j}$, for example, $G_h \phi = 0$ while $L_5 \phi = -8\phi$. This illustrates the grid-scale decoupling possible with collocated central differences; it is a mathematical example, not a claim that a particular app frame contains that pattern.

Boundaries, convergence, and conservation

Clamped pressure sampling resembles a zero-normal-gradient extension. Such Neumann-like systems leave the constant pressure offset undetermined. The code does not pin a pressure value, subtract its mean, or explicitly enforce the right-hand-side compatibility condition. Velocity depends on gradients, but the pressure visualization can still depend on its offset.

Jacobi is local and simple to parallelize, but long-wavelength error relaxes slowly. A fixed count gives predictable work, not grid-independent accuracy; larger grids do not receive more iterations. Half-precision arithmetic further limits small corrections. The final zero-velocity border adds another source of discrete divergence near the wall.

What is sacrificed for visualization. The scheme does not promise exact dye-mass conservation, exact incompressibility, momentum conservation, or energy conservation. Advection interpolation, retention, user injection, confinement, wall overwrites, and half-precision storage all affect these quantities. There is no quantitative error tolerance or adaptive refinement.

These limitations do not prevent compelling swirling dye. They do mean that visual plausibility must not be interpreted as a validated prediction of a physical flow. A consistent discrete projection, convergence monitoring, and a specified physical timestep would be separate numerical design changes.

Source: Algebra derived from the divergence, jacobi, and gradient stencils; no empirical error measurement is asserted.

8 Making each step practical on the GPU

Textures as arrays; drawing as computation

Each slab owns two `rg16Float` textures, called `ping` and `pong`. The renderer draws a full-screen quad (two indexed triangles) into `pong`; a fragment invocation computes each destination cell from `ping`. Swapping the references makes the output the next input. This avoids reading and writing the same texture in one pass and makes Jacobi’s iteration dependency explicit. The active simulation uses render passes, not the project’s separate `ComputeShader` helper.

With B_c contact batches and an available drawable, an ordinary advancing frame uses

$$\underbrace{2}_{\text{advection}} + \underbrace{2B_c}_{\text{input}} + \underbrace{2}_{\text{curl + confinement}} + \underbrace{1}_{\text{divergence}} + \underbrace{40}_{\text{pressure}} + \underbrace{1}_{\text{gradient}} + \underbrace{1}_{\text{display}} = 47 + 2B_c \quad (24)$$

render passes. With one contact batch this is 49. Most work is local texture sampling; the 40 pressure passes dominate the pass count. The work scales roughly as $O(WH(40 + B_c + 1))$, with up to ten splats evaluated per input pass. This is a cost model, not a measured frame-rate claim.

Five double-buffered slabs at two 16-bit channels per texel require nominally $5 \times 2 \times 4WH = 40WH$ bytes of field storage, excluding alignment, uniforms, drawables, and transient resize textures. Scalars still occupy two channels. Half storage reduces memory traffic relative to 32-bit channels but has limited range and precision. Some shaders calculate in `float`; splats, much of Jacobi, and final outputs use `half`. Rounding occurs between passes.

Initial values and scheduling

`Slab.init` allocates textures but performs no explicit clear or upload. `RenderShader` sets `loadAction = .dontCare`; its black `clearColor` is therefore not an initialization pass. From source alone, one should not claim a solver-defined zero initial condition. A reproducible numerical experiment would explicitly initialize all fields before sampling them. The warm-start description applies once a pressure result has been stored.

Three uniform buffers and a semaphore permit up to three frames in flight, while command buffers use the same command queue. This is scheduling overlap; it does not mean three independent fluid states are simulated.

Resizing preserves an image-space state

The resize pass nearest-samples each old field at normalized coordinates. For velocity only, it applies componentwise scale factors,

$$u_{\text{new}}(\boldsymbol{\xi}) = \frac{W_{\text{new}}}{W_{\text{old}}} u_{\text{old}}(\boldsymbol{\xi}), \quad v_{\text{new}}(\boldsymbol{\xi}) = \frac{H_{\text{new}}}{H_{\text{old}}} v_{\text{old}}(\boldsymbol{\xi}). \quad (25)$$

This preserves displacement as a fraction of the canvas per step. Other fields are copied without amplitude scaling. No conservative remap or immediate reprojection occurs; copied divergence/vorticity need not be derivatives of the resized velocity. They are recomputed on the next advancing frame. Input is cleared and new grid offsets are installed.

Source: `Slab.swift`, `RenderShader.swift`, `Renderer.swift`, `MetalDevice.swift`; `Shaders.metal`: `resampleField`.

9 What the controls and colours mean

The displayed field is not a calibrated plot

Density and Pressure use the first texture channel q and map it to

$$(R, G, B, A) = (0, 0.06|q|, 0.19|q|, 1). \quad (26)$$

The absolute value discards pressure sign. Brightness is not normalized to the range of the field. On the `bgra8Unorm` output target, channels clamp to $[0, 1]$ and are quantized, so blue saturates at $|q| \simeq 5.26$ and green at $|q| \simeq 16.67$ in ideal arithmetic.

Velocity and Vorticity use `visualizeVector`, whose expression is

$$\mathbf{C} = \frac{1}{2}\mathbf{1} + \frac{1}{2}\mathbf{q}. \quad (27)$$

Velocity therefore places u in red and v in green, centred on mid-grey for zero components; it is not a vector-arrow or speed-magnitude plot. Vorticity is actually scalar, stored as $(\omega, 0)$, so its red channel varies with signed ω and its green channel stays at 0.5. Values outside $[-1, 1]$ clip in these displayed channels. The same sample conversion also handles the unused blue/alpha components.

The stored vorticity was computed *before* confinement and projection in the frame. It is not a recomputed curl of the final displayed velocity. Divergence has no selectable display field, and selecting a field changes only the presentation, not the evolving simulation.

Slider position versus numerical value

For normalized slider position $z \in [0, 1]$, Force, Dye, and Swirl each map to $2z$. Fade maps to retention by the following piecewise rule:

$$r(z) = \begin{cases} 0.9995 - 0.0015(z/0.17), & z < 0.17, \\ 0.998, & z = 0.17, \\ 0.998 - 0.0075((z - 0.17)/0.83), & z > 0.17. \end{cases} \quad (28)$$

The central equality is explicitly special-cased in Swift. The UI range is $0.9905 \leq r \leq 0.9995$: even minimum Fade retains less than one, and maximum Fade does not immediately erase a field.

Control	Default UI	Default value	Effect
Force	50%	$F = 1$	Multiplies contact displacement.
Dye	40%	$d = 0.8$	Adds dye per contact per advancing frame.
Swirl	20%	$S = 0.4$	Scales confinement of existing vorticity.
Fade	17%	$r = 0.998$	Retains velocity and dye during advection.

Radius and pressure-iteration properties exist in `SimulationTuning`, but the live pipeline chooses radius through its input paths and hard-codes 40 iterations. There is no runtime resolution control. Changing the source grid scale alone would also require care with force, splat width, timestep, and projection accuracy if consistent physical behaviour were the goal.

Source: `SimulationState.swift`: `SimulationTuning`; `Renderer`: `render`, `drawSlab`; `visualizeScalar`, `visualizeVector`.

10 Source map and reproducibility

All paths below are relative to the repository root. Function names are given instead of line numbers so that small unrelated edits do not invalidate the map. The description was checked against simulation sources at commit `9477dabc41a24780371c07e8dec7a9ba3d337a0`.

File	What to inspect
<code>Sources/Shared/Renderer.swift</code>	<code>draw(in:)</code> for order and 40 iterations; <code>writeContacts/nextBuffer</code> for input; resize callback for grid size and remapping; <code>render</code> for field-to-shader selection.
<code>Sources/Shared/Shaders.metal</code>	<code>advect/bilerpFrag</code> ; force shaders; <code>vorticity/vorticityConfinement</code> ; <code>divergence/jacobi/gradient</code> ; visualizers and <code>resampleField</code> . These define the discrete equations.
<code>Sources/Shared/Slab.swift</code>	Texture formats, paired allocation, and swapping.
<code>Sources/Shared/RenderShader.swift</code>	Full-quad render passes and load/store actions.
<code>Sources/Shared/MetalDevice.swift</code>	Shared device, queue, and pipeline creation.
<code>Sources/Shared/SimulationState.swift</code>	Slider mappings, defaults, and advance/pause state.
<code>Sources/macOS/RenderViewController.swift</code>	Mouse position conversion and forwarding.
<code>Sources/iOS/RenderViewController.swift</code>	<code>submitTouches</code> displacement construction; tap and hold handling.

Evidence and limits

The equations here are transcriptions or algebraic consequences of the source, with ordinary fluid mechanics used to explain their purpose. The document does not report a new GPU benchmark, conservation test, or convergence study. Existing `Tests/Integration/test_phase05_metal.swift` exercises shader responses to force, dye, retention, and swirl; `test_phase03_metal.swift` in the same directory exercises resize remapping. Those checks are not a proof of Navier–Stokes accuracy or discrete incompressibility.

Optional primary references

- Apple, *Metal Shading Language Specification*, 4 June 2026, section 2.10, table 2.7: sampler defaults used on page 6. [Official specification PDF](#).
- Mark J. Harris, *GPU Gems*, chapter 38, “Fast Fluid Dynamics Simulation on the GPU” (2004). [Current NVIDIA chapter page](#). This is background reading; its algorithm should not be substituted for this repository’s exact implementation.

Rebuild the document

The editable source and generated PDF live together in `docs/`. From the repository root, with a LaTeX distribution providing the packages named in the preamble, run:

```
latexmk -pdf -interaction=nonstopmode -halt-on-error \
  -outdir=docs docs/fluid-dynamics.tex
```